1 Постановка задачи

В рамках задания требовалось построить небольшую авторегрессионную языковую модель с нуля для языка, не входящего в число 50 наиболее распространённых. Для проекта GeorgiaGPT использован грузинский язык. Технической постановкой предусмотрена реализация decoder-only Transformer на PyTorch.

2 Подготовка корпуса

2.1 Источник данных и разбиение

В качестве источника выбран грузинский сегмент `kat_Geor` набора FineWeb2. Применение уже подготовленного веб-корпуса позволило сосредоточиться на построении собственной модели, не выполняя отдельный сбор страниц и разработку всей системы очистки веб-текстов.

Исходный массив содержит 3 706 659 документов в двух файлах Parquet. Документы были распределены на обучающую и валидационную части в пропорции 99:1 с `seed=42`: 3 669 592 и 37 067 документов соответственно. Разбиение производилось по индексам исходных документов. Индексы валидационных документов и ревизия FineWeb2 сохранены в манифесте.

2.2 Разведочный анализ текста

Разведочный анализ (EDA) выполнялся по всему обучающему корпусу, потоково. Для каждого документа вычислялись длина в символах и словах, число строк, доля букв разных письменностей, признаки повторяющихся строк и некоторые характеристики веб-метаданных. Промежуточные статистики агрегировались по мере чтения Parquet.

Таблица 1: Основные характеристики корпуса по результатам EDA

Показатель	Среднее	Стандартное отклонение
Символов на документ	2771	5948
Слов на документ	330,6	748,3
Строк на документ	15,84	43,38
Доля грузинских букв	0,9725	0,0724

Средняя длина около 331 слова при стандартном отклонении более 748 слов показывает, что корпус неоднороден: наряду с короткими фрагментами присутствуют длинные веб-страницы. Поэтому одного среднего значения недостаточно для описания данных. Распределение длин на рис. 1 помогает увидеть масштаб неоднородности; этот результат также объясняет выбор обучения на окнах фиксированной длины вместо попытки обрабатывать каждый документ целиком.

Средняя доля грузинских букв составляет около 97,25% среди буквенных символов. Высокая концентрация нужной письменности подтверждает пригодность сегмента для базового обучения модели. Небольшие включения латиницы, цифр и технических строк при этом не обязательно явля-

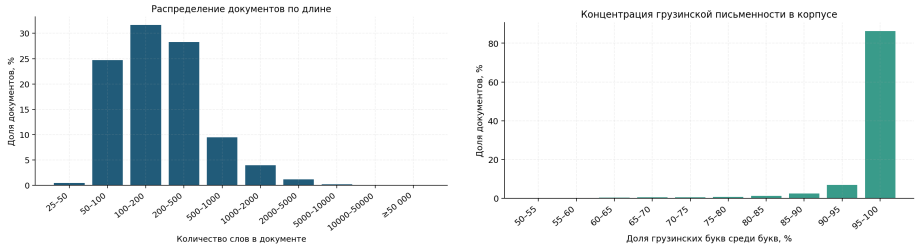


Рис. 1: Распределение длин документов (слева) и доли грузинских букв (справа) по полному корпусу.

ются ошибками: они могут обозначать имена, ссылки и элементы исходной веб-разметки. По этой причине не вводился дополнительный жёсткий порог удаления документов только по доле грузинских символов.

Метаданные Common Crawl распределены по годам веб-снимков 2013–2024 (рис. 2). Год снимка отражает время сбора страницы, а не обязательно дату написания текста. Эта характеристика использовалась для описания происхождения данных. После EDA отдельная фильтрация корпуса в рамках проекта не выполнялась: учитывая происхождение этого набора данных и полученных результатов анализа причин для дополнительной обработки корпуса не было выявлено.



Рис. 2: Распределение документов по годам веб-снимков Common Crawl.

3 Токенизация и представление данных

3.1 Использование byte-level BPE

В проекте использован byte-level BPE: он сохраняет возможность кодировать произвольный текст и выделяет часто встречающиеся подсловные последовательности. Для грузинского языка, в котором встречаются разнообразные словоформы, такое представление позволяет ограничить размер словаря без потери охвата.

В такой схеме текст сначала представляется байтами UTF-8, после чего частые соседние последовательности объединяются в подсловные единицы. Неизвестное слово можно закодировать без отдельного токена `UNK`. Основной компромисс связан с размером словаря: более крупный словарь уменьшает число токенов на текст, но увеличивает матрицу эмбеддингов и, следовательно, число параметров модели. При контекстном окне в 512 позиций компактность кодирования особенно важна: чем меньше токенов требуется для фразы, тем больший объём текста помещается в доступный контекст.

3.2 Выбор размера словаря

Для проверки компромисса обучены шесть токенизаторов: словари на 8192 и 16 384 токена, каждый на 10, 30 и 100 тыс. train-документов. Они сравнивались на одной фиксированной выборке из 2000 документов validation, не участвовавших в обучении BPE. Основные измерения приведены в табл. 2; тот же результат показан на рис. 3.

Таблица 2: Эффективность BPE на 2000 validation-документов

Документов	Словарь	Токенов/слово	Ток./1000 симв.
10 000	8192	2,2487	268,63
10 000	16 384	1,9598	234,12
30 000	8192	2,2446	268,13
30 000	16 384	1,9540	233,42
100 000	8192	2,2441	268,08
100 000	16 384	1,9524	233,23

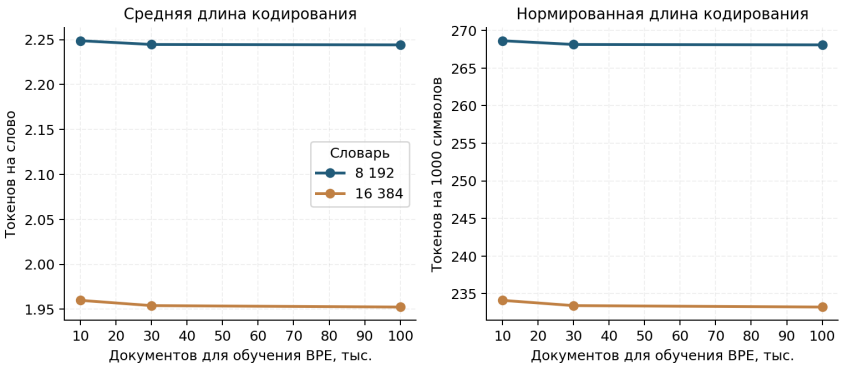


Рис. 3: Число токенов на слово и на 1000 символов для шести вариантов BPE.

По сравнению с 8192 единицами словарь 16 384 сокращает длину представления примерно на 13% при обучении на 100 тыс. документов.

Окончательно выбран словарь $V = 16\,384$, обученный на 100 тыс. train-документов. Выбранный вариант обеспечивает более короткие последова-

тельности, сохраняя размер словаря достаточно небольшим для моделей порядка 40–50 млн параметров.

3.3 Формат обучающей выборки

После фиксации токенизатора весь корпус был закодирован в `train.bin` и `val.bin`. Использован формат `uint16`: он достаточен для хранения идентификаторов словаря из 16 384 токенов и занимает вдвое меньше места, чем `uint32`. Специальные идентификаторы — `BOS=0`, `EOS=1`, `PAD=2`. В результате получено 2 370 606 697 токенов train (около 4,74 ГБ) и 24 205 290 токенов validation (около 48,4 МБ).

Такой формат выбран ради простого и быстрого чтения. Вместо разбора текстовых документов при каждом шаге обучения программа использует memory mapping и извлекает короткие диапазоны токенов по индексам. Это сокращает затраты на подготовку мини-батчей и не требует загружать весь корпус в оперативную память. Манифест токенизации сохраняет идентичность исходных данных, контрольную сумму токенизатора, число токенов и порядок байтов.

4 Архитектура GeorgiaGPT

4.1 Общая схема и размер моделей

Реализована decoder-only архитектура Transformer, в которой каждая позиция использует только предшествующие токены. Для последовательности $x_1, \ldots, x_T$ модель обучается предсказывать следующий токен с условной вероятностью $p_\theta(x_t \mid x_{<t})$. Функция потерь представляет собой среднюю отрицательную логарифмическую вероятность правильных токенов:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{t=1}^{N} \log p_\theta(x_t \mid x_{<t}). \tag{1}$$

Входные индексы преобразуются в эмбеддинги размерности 512, далее проходят через несколько блоков Transformer, финальную RMSNorm и линейную проекцию в словарь. Предусмотренное архитектурой связывание входных и выходных весов (weight tying) позволяет использовать одну матрицу для обеих операций. Для данного словаря и размерности отдельная выходная матрица $V \times d$ потребовала бы ещё 8,39 млн параметров.

Две конфигурации имеют одинаковый словарь, длину контекста и размерность скрытых представлений. Различаются только глубина сети и ширина полносвязного блока. Это позволяет отдельно рассмотреть влияние увеличения ёмкости модели, не меняя способ кодирования исходных текстов. Выбранные размеры сохраняют вычислительную задачу осуществимой на одной CUDA-системе.

4.2 Внимание, позиции и нормализация

Заданная архитектура использует Causal Multi-Head Self-Attention; в реализованных конфигурациях предусмотрены 8 голов по 64 компоненты. Причинная маска запрещает обращаться к будущим позициям, обеспечивая

Таблица 3: Конфигурации моделей GeorgiaGPT

Параметр	40M	49M
Число блоков	10	12
Скрытая размерность	512	512
Голов внимания	8	8
Промежуточная ширина SwiGLU	1376	1536
Контекст, токенов	512	512
Dropout	0	0
Обучаемых параметров	40 020 480	49 295 872

авторегрессионное предсказание. Конкретный способ вычисления внимания выбран на уровне реализации: функция `scaled_dot_product_attention` из PyTorch позволяет задействовать оптимизированные CUDA-ядра, когда они доступны.

Было использовано позиционное кодирование RoPE. Оно выполняет поворот пар координат запросов и ключей на углы, зависящие от позиции токена. Благодаря этому относительное смещение учитывается в скалярных произведениях внимания без отдельной обучаемой таблицы позиционных эмбеддингов. В реализации использовано основание RoPE $\theta = 10\,000$.

RMSNorm применяется до внимания и полносвязного преобразования (pre-normalization), а результаты подблоков складываются с их входами через остаточные соединения. Такая организация помогает распространению градиентов по глубине сети. RMSNorm нормирует масштаб вектора без вычитания среднего. Для вычисления нормы в реализации используется промежуточная точность FP32 и $\varepsilon = 10^{-6}$.

Полносвязная часть представляет собой SwiGLU:

$$\mathrm{SwiGLU}(x) = W_{\mathrm{down}}\big(\mathrm{SiLU}(W_{\mathrm{gate}}x) \odot W_{\mathrm{up}}x\big). \tag{2}$$

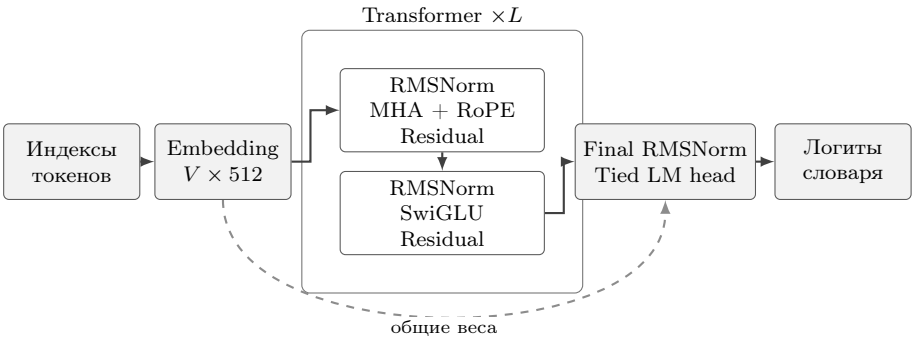


Рис. 4: Укрупнённая схема GeorgiaGPT: стек из L блоков Transformer, два остаточных подблока и связанная выходная проекция.

5 Организация обучения

5.1 Мини-батчи и оптимизация

При длине контекста 512 обучающий пример представляет собой окно из 513 последовательных токенов: 512 входных и 512 целевых позиций, сдвинутых на один токен. Окна выбираются из единого бинарного потока; для train используется воспроизводимый случайный семплер. Одинаковая длина окон упрощает формирование мини-батчей и не требует дополнения коротких документов до общей длины, как при работе с отдельными текстами.

Использован оптимизатор AdamW с начальным learning rate $2 \cdot 10^{-4}$, $\beta_1 = 0{,}9$, $\beta_2 = 0{,}95$, weight decay 0,1 и ограничением нормы градиента до 1,0. AdamW выбран как стандартный оптимизатор Transformer, позволяющий использовать адаптивные оценки моментов градиента и отдельно задавать уменьшение весов. Ограничение нормы уменьшает влияние единичных резких градиентных обновлений.

Для уменьшения пиковой нагрузки на GPU использовались микробатчи по четыре последовательности и накопление градиентов за четыре прохода. В результате одно обновление параметров соответствует $4 \times 4 \times 512 = 8192$ токенным позициям, но все 16 последовательностей не требуется одновременно держать в памяти.

Обучение моделей GeorgiaGPT проводилось локально на видеокарте NVIDIA GeForce RTX 5060 Laptop GPU с 8 ГБ видеопамяти. Использовались PyTorch 2.13.0, CUDA 13.0 и смешанная точность BF16, позволявшая снизить потребление видеопамяти при вычислениях.

Таблица 4: Основные параметры обучения

Параметр	Значение	Параметр	Значение
Microbatch	4	Накопление	4
Контекст	512	Токенов/шаг	8192
Начальный LR	$2 \cdot 10^{-4}$	Warmup	400 шагов
Weight decay	0,1	Gradient clip	1,0
Validation	500 шагов	Checkpoint	1000 шагов
Точность	BF16	Train-лог	20 шагов

5.2 Последовательные стадии и checkpoint

Начальный планировщик learning rate использует прогрев в течение 400 шагов и затем косинусное снижение. Прогрев уменьшает риск слишком крупных обновлений в начале, когда оценки моментов AdamW ещё формируются; последующее снижение позволяет постепенно уменьшать размер обновлений. Длительное обучение разделялось на самостоятельные стадии, чтобы сохранять промежуточные состояния и продолжать эксперименты без повторного запуска с нуля.

Конфигурация 40M прошла начальную стадию до 60 тыс. шагов и продолжение до 200 тыс. Конфигурация 49M обучалась до 100 тыс. шагов, затем последовательно продолжалась до 200, 300 и 400 тыс. шагов. На каждое обновление приходилось 8192 токенных позиции, поэтому суммарный

обработанный объём составил 1,6384 млрд и 3,2768 млрд токенных позиций соответственно. Это число включает повторные обращения к корпусу.

Для промежуточных стадий реализован режим `extend`: обучение продолжается из завершённого checkpoint, но новая стадия получает отдельную конфигурацию и каталог результатов. В отличие от `resume`, который восстанавливает тот же незавершённый запуск вместе с оптимизатором, scheduler, состоянием семплера и RNG, `extend` даёт возможность изменить расписание LR без перезаписи предшествующей истории.

Для продолжений до 200 тыс. шагов использовался переход к косинусному расписанию с пиком LR $6 \cdot 10^{-5}$ и конечным значением 10^{-5}. В продолжении 49M до 300 тыс. LR снижался с 10^{-5} до $5 \cdot 10^{-6}$, затем до 400 тыс. — с $5 \cdot 10^{-6}$ до $2{,}5 \cdot 10^{-6}$. Фактическая динамика расписания показана на рис. 5.

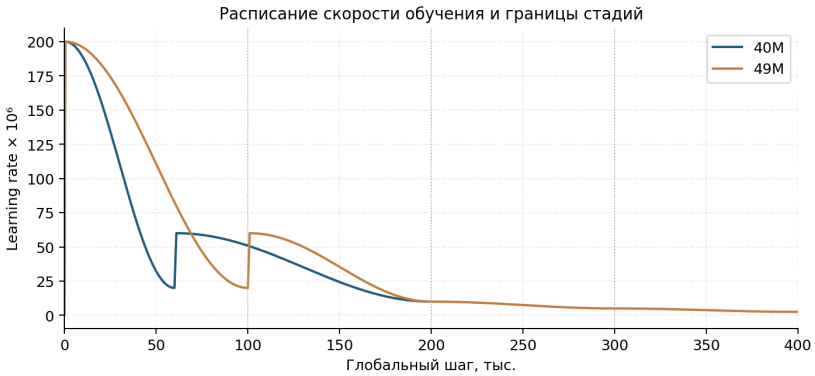


Рис. 5: Learning rate по глобальному шагу обучения, включая продолжения 40M и 49M.

5.3 Мониторинг обучения

Каждые 20 шагов сохранялись обучающие метрики, каждые 500 шагов рассчитывалась validation loss на фиксированных 256 окнах (131 072 целевых токена), каждые 1000 шагов сохранялся checkpoint. Фиксированная валидация выбрана для сопоставимости измерений между стадиями: при постоянно меняющихся случайных окнах часть колебаний loss отражала бы не модель, а состав проверочных данных. Вместе с функцией потерь регистрировались норма градиента, скорость обработки токенов и использование памяти CUDA.

Графики рис. 6 и 7 построены по журналам `metrics.jsonl`. Обучающая кривая сглажена только для отображения, validation loss показана без сглаживания.

6 Воспроизводимость

Для корпуса, разбиения и токенизатора сохранены манифесты и контрольные суммы. Каждый тренировочный запуск сопровождается конфигурацией, checkpoint и журналом `metrics.jsonl`. Это позволяет продолжать

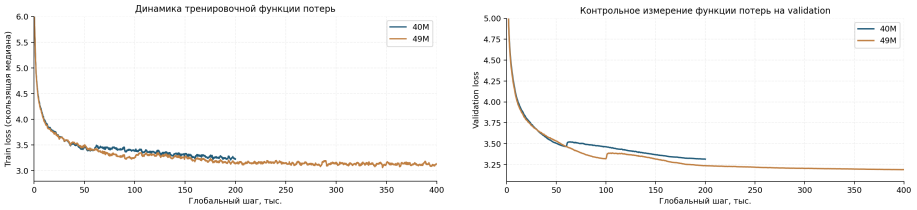


Рис. 6: Динамика train loss (слева) и validation loss (справа).

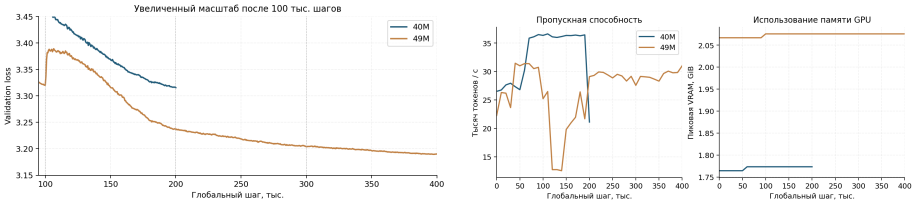


Рис. 7: Validation loss на поздних этапах (слева) и вычислительные характеристики обучения (справа).

прерванное обучение, связывать результаты с конкретными настройками и повторно строить графики без загрузки весов модели.

7 Оценка языковой модели

7.1 Benchmark v3

Для сравнения пяти сохранённых моделей использован Benchmark v3 из 54 грузиноязычных заданий: 18 продолжений текста разных жанров, 18 дополнений о фактах Грузии и 18 заданий на использование контекста (12 дополнений и 6 вопросов). В последней группе предусматривался контрольный промпт без нужного факта. Проверка проводилась на одних и тех же заданиях в пяти режимах: `greedy`, `topk_t08` ($k = 40$, $T = 0{,}8$), `topp_t08` ($p = 0{,}9$, $T = 0{,}8$), `conservative` ($p = 0{,}9$, $T = 0{,}65$) и `diverse` ($p = 0{,}95$, $T = 1$). Всего получено 1350 генераций.

Для фактических и контекстных заданий дополнительно вычислялся ранг правильного ответа среди четырёх альтернатив по суммарной логарифмической вероятности токенов. Ранг 1 соответствует предпочтению правильного варианта. В контекстной группе сравнивались ранги и вероятности с опорным фактом и без него. Показатель характеризует выбор среди предложенных вариантов, а не правильность свободной генерации; различия в токенной длине альтернатив также влияют на оценку.

7.2 Качественная оценка

Для оценки естественности текста отдельно отобраны 450 свободных продолжений. Их анализ выполнялся моделью-судьёй **GPT Astra, режим Medium**: пять критериев по шкале 0–2 (начало, язык, тема и

жанр, связность, устойчивость) и общая категория «удачно», «частично удачно» либо «неудачно». Тексты передавались пакетами по 15 без названий моделей и режимов. Точная инструкция приводится ниже; подробные результаты, визуализации и примеры с переводом находятся в `notebooks/03_comparison.ipynb`. Там приведен подробный анализ и учитывая объем, было решено не переносить выводы сюда.

7.3 Инструкция модели-судье

GeorgiaGPT — независимая качественная оценка продолжений
Ты выступаешь в роли независимого эксперта по грузинскому языку. Твоя задача — оценить продолжения грузинских текстов, сгенерированные небольшой базовой языковой моделью (не instruction-tuned).
Я буду присылать JSON-файлы по 15 записей. Каждый файл содержит `batch_id` и массив `items`. У каждой записи есть `item_id`, `prompt` (начало текста) и `generated_text` (только сгенерированное продолжение).
Оценивай **только** `generated_text` **в контексте** `prompt`. Не оценивай сам промпт и не считай исходный промпт частью продолжения. Эталонного текста нет: любой естественный и осмысленный вариант допустим. Не требуй соблюдения инструкций или литературного совершенства. Не пытайся определить модель, режим декодирования или сравнивать записи между собой. Применяй одни и те же критерии ко всем записям.
Критерии (0, 1, 2)

- `beginning`: насколько первые 1–2 содержательные фразы естественно и грамматически уместно продолжают промпт.
- `language`: грузинская орфография, морфология, падежи, синтаксис и естественность словосочетаний во всём продолжении.
- `topic_genre`: сохранение темы, жанра и стиля, заданных промптом.
- `coherence`: логическая связность, последовательность мыслей и событий, отсутствие смысловых противоречий.
- `stability`: отсутствие зацикливания, механических повторов, текстового мусора и резкого ухудшения к концу текста.

Для каждого критерия: `2` — хорошо; `1` — частично; `0` — плохо. `null` допускается только тогда, когда критерий невозможно оценить по представленному тексту; объясни причину в `reason_ru`.
Общая категория
Категория `overall` не вычисляется простой суммой пяти баллов:

- `"Удачно"`: большая часть продолжения естественна, понятна, связна и соответствует промпту.
- `"Частично удачно"`: есть достаточно содержательный удачный фрагмент, но встречаются заметные ошибки, повторы либо потеря темы.
- `"Неудачно"`: содержательное продолжение почти отсутствует или текст сразу/очень быстро становится бессвязным.

Если начало хорошее, а затем качество заметно падает, не обнуляй оценку начала; обычно итог — `"Частично удачно"`, если начальный фрагмент действительно содержателен. Отсутствие повторов не означает смысловой связности. Не путай языковые, логические и фактические ошибки. Если сомневаешься в конкретной грузинской форме, укажи неопределённость вместо уверенного утверждения.
Отдельно укажи `good_beginning` (было ли хорошее содержательное начало) и `later_degradation` (наблюдается ли явная деградация после начала) как `true` или `false`.
`evidence_good` и `evidence_problem` — короткие **дословные** цитаты только из `generated_text`, подтверждающие оценку, либо `null`. Не придумывай цитаты.
`reason_ru` — краткое конкретное обоснование на русском (1–3 предложения). Не делай перевод всего текста.
Формат ответа — строго JSON
На каждый полученный файл верни **один валидный JSON-объект**, без Markdown, тройных кавычек, вступления, таблиц, статистики и сводных выводов. Структура:

```
{
  "batch_id": "batch_001",
  "judgments": [
    {
      "item_id": "j_example123",
      "scores": {
        "beginning": 2,
        "language": 1,
        "topic_genre": 1,
        "coherence": 1,
```

```
      "stability": 0
    },
    "overall": "Частично удачно",
    "good_beginning": true,
    "later_degradation": true,
    "evidence_good": "Точная короткая цитата из generated_text или null",
    "evidence_problem": "Точная короткая цитата из generated_text или null",
    "reason_ru": "Краткое предметное обоснование на русском."
  }
 ]
}
```

Пример показывает **структуру**, но не готовую оценку какого-либо текста. Для каждого `item_id` из входного массива `items` создай ровно одну запись в `judgments` и **сохрани** `item_id` **и** `batch_id` **без изменений**. Количество записей `judgments` должно совпадать с количеством `items`; порядок сохраняй. Все пять ключей в `scores` обязательны. Используй только целые числа 0, 1, 2 или `null`, только три разрешённые строки `overall` и настоящие JSON-значения `true`, `false`, `null`.

Не формируй общую статистику: агрегирование по моделям, заданиям и режимам декодирования будет выполнено отдельно после объединения всех ответов.

8 Итоги

В ходе работы подготовлен грузинский корпус, обучен собственный byte-level BPE-токенизатор, реализована модель GeorgiaGPT и проведено её обучение в конфигурациях порядка 40 и 49 млн параметров. Для оценки создан Benchmark v3 с заданиями на факты, контекст и свободную генерацию.

Полученная модель смогла воспроизводить отдельные сведения о Грузии и создавать естественно звучащие короткие фрагменты на грузинском языке. На большей дистанции качество часто снижалось: появлялись повторы и нарушалась связность. Для инференса выбрана модель 49M/300k с Top-k ($k = 40$, $T = 0{,}8$), используемая в `04_inference.ipynb`.